Behavioral Segmentation for Targeted Marketing

ProjectId:PY-PT25

Name :TANISH SOOD (Team Leader)

Class Roll Number: 25

University Roll Number : 2028508

Section and Group: P (G1)

Name :SAMBHAWI PANDEY

Class Roll Number: 18

University Roll Number : 2028481

Section and Group: P (G1)

Name :CHIRAG VARSHNEY

Class Roll Number: 38

University Roll Number : 2028694

Section and Group: P (G1)

TABLE OF CONTENTS

1. Problem Description

The Behavioral Segmentation for Targeted Marketing system is a Python-based machine learning application developed to automate the process of grouping retail customers into distinct behavioral personas and generating personalized marketing strategies for each group. It streamlines the process of collecting transaction-level customer data, engineering behavioral features, training clustering and classification models, and producing actionable discount recommendations — all while maintaining high accuracy and interpretability.

Using core concepts of Machine Learning such as data preprocessing, feature engineering, unsupervised clustering (K-Means and Hierarchical), supervised classification (KNN), model evaluation through Silhouette Score, and 2D/3D visualization via PCA, this project minimizes the manual effort required for customer profiling and supports data-driven targeted marketing decisions for retail businesses.

The system goes beyond the base project requirement of K-Means + Elbow visualization by introducing an original innovation: a trained KNN classifier that can predict the customer segment of any **new customer** in real time, based on just three input values. This makes the pipeline production-ready and applicable beyond the training dataset.

1.1 Dataset Description

The dataset used for this project is a custom retail transaction dataset named FINAL_PBL_DATA_SET_7.csv containing 238,903 transaction records and 10 attributes, collected to analyze customer purchasing behavior and retail transaction patterns. The dataset includes customer information, invoice details, product data, transaction quantity, pricing information, and product return records.

The dataset consists of transaction-based records where each row represents one line item from a customer invoice.

The dataset contains both numerical and categorical attributes. Numerical attributes include CUSTOMER_ID, QUANTITY, UNIT_PRICE, and NO_OF_ITEMS_RETURNED. Categorical attributes include INVOICE_NO, STOCK_CODE, PRODUCT_CATEGORY, and ITEM_RETURNED. The dataset also includes date-time information through the INVOICE_DATE attribute for transaction time analysis.

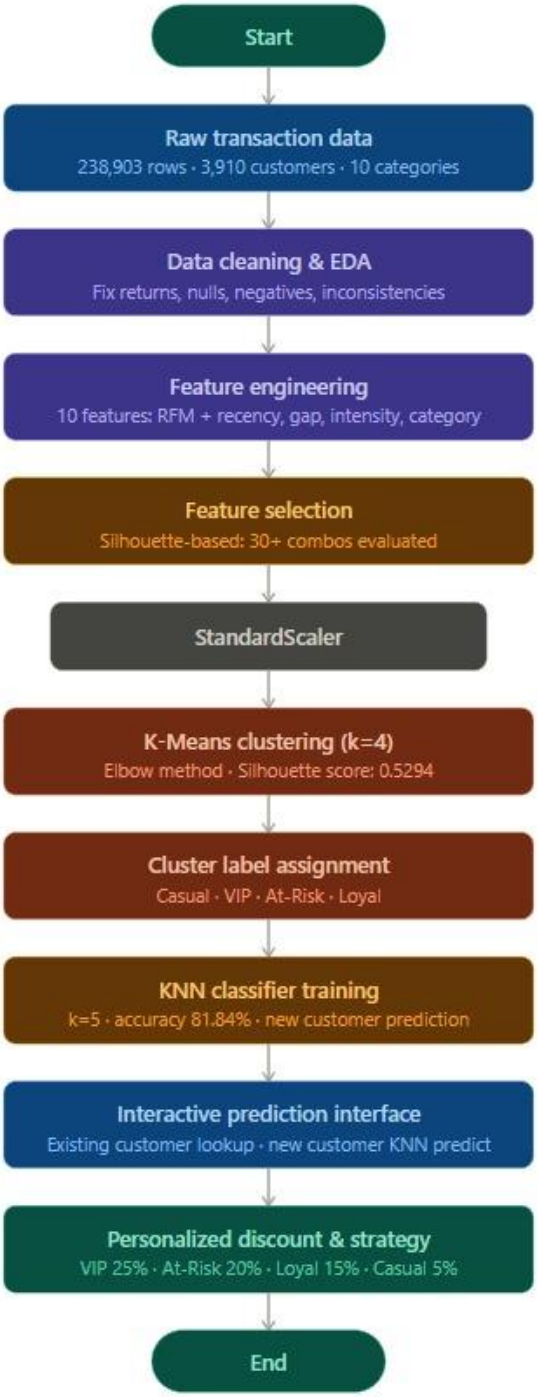
The major attributes used in the dataset are:

• CUSTOMER_ID: Unique identification number assigned to each customer
• INVOICE_NO: Unique invoice number for each transaction
• STOCK_CODE: Product SKU code of the purchased item
• QUANTITY: Number of units purchased in the transaction
• INVOICE_DATE: Date and time of the transaction
• UNIT_PRICE: Price per unit of the product in Indian Rupees
• PRODUCT_CATEGORY: Category of the purchased product
• ITEM_RETURNED: Indicates whether the item was returned (YES/NO)
• NO_OF_ITEMS_RETURNED: Number of items returned from the transaction

The dataset contains transactions from 3,910 unique customers with customer IDs ranging from 12,747 to 18,287. Products are classified into ten categories including Sports, Beauty, Gifts, Electronics, Stationery, Kitchenware, Home Decor, Toys, Groceries, and Clothing.

Approximately 10.1% of the transaction rows involve returned items, with a total of 24,174 returned line items recorded across 3,197 unique customers.

1.2 Flow Diagram



2. Modules for Project Implementation

2.1 Modules Used

• **pandas** : Used for data loading, cleaning, manipulation, and aggregation.
Functions used:

• **numpy** : Used for numerical and mathematical operations.
Functions used:

`np.where()` for outlier clipping
`np.sqrt()` for feature transformation
`np.array()` for model input preparation

• **scikit-learn (sklearn)** : Used for machine learning, preprocessing, clustering, and evaluation.
Functions used:

`StandardScaler` for feature normalization
`KMeans` and `AgglomerativeClustering` for clustering
`KNeighborsClassifier` for prediction
`PCA` for dimensionality reduction
`silhouette_score` for cluster evaluation
`train_test_split()` for data splitting
`accuracy_score` and `classification_report` for model evaluation
`LabelEncoder` for target encoding

• **matplotlib** : Used for data visualization and plotting graphs.
Functions used:

`plt.plot()` for Elbow method graph
`plt.scatter()` for PCA cluster plots
`plt.figure()` and `plt.show()` for visualization rendering

• **seaborn** : Used for statistical and advanced visualizations.
Functions used:

`sns.histplot()` for distribution plots
`sns.boxplot()` for cluster comparison

`sns.barplot()` for feature analysis
`sns.scatterplot()` for cluster visualization

• **itertools** : Python built-in library used for feature combination generation.
Functions used:

`itertools.combinations()` for exhaustive feature selection

2.2 Platform Used

Jupyter Notebook : Used for all six stages of development (data cleaning, feature engineering, feature selection, K-Means clustering, hierarchical clustering, and final KNN model). Jupyter was chosen for its cell-by-cell execution model, which allows incremental development, inline visualization of plots, and easy inspection of intermediate dataframes.

Python 3.13 : Core programming language used throughout the project for data processing, model training, and the interactive prediction interface.

VS Code : Used as a supplementary editor for writing and organizing Python scripts.

3. Machine Learning Models / Techniques Used

K-Means Clustering

K-Means is an unsupervised machine learning clustering algorithm used to divide data points into k different groups or clusters based on similarity. Since it is an unsupervised algorithm, it does not require predefined labels or output classes. The main goal of K-Means is to group similar data points together while keeping different groups as separate as possible.

The algorithm works in multiple steps. First, the number of clusters k is selected. Then, k random centroids (center points) are initialized. Each data point is assigned to the nearest centroid based on distance calculations, usually Euclidean distance. After assigning all points, the centroids are recalculated by taking the average of all points within each cluster. This process of assigning points and updating centroids repeats until the centroids no longer change significantly and stable clusters are formed.

The objective of K-Means is to minimize the Within-Cluster Sum of Squares (WCSS), which measures the total squared distance between data points and their cluster centroids. Lower WCSS values indicate better and more compact clustering.

Formula for WCSS:

$$WCSS=\sum_{i=1}^{k}\sum_{x\in C_i}(x-\mu_i)^2$$

Where:
- k = number of clusters
- C_i = data points belonging to cluster i
- μ_i = centroid of cluster i
- $(x - \mu_i)^2$ = squared distance between a data point and its centroid

In this project, K-Means clustering was applied on the scaled features TOTAL_SPENDING, FREQUENCY, and AVG_PURCHASE_GAP to segment customers based on their purchasing behavior. Before clustering, the features were normalized using StandardScaler to ensure equal contribution of each feature during distance calculation.

The Elbow Method was used to determine the optimal number of clusters, and the best value was found to be k=4. The model achieved a Silhouette Score of 0.5294, indicating good cluster separation and compactness. Based on the clustering results, customers were grouped into four meaningful segments: Casual Customers, VIP Customers, At-Risk Customers, and Loyal Customers.

Agglomerative Hierarchical Clustering

Agglomerative Hierarchical Clustering is an unsupervised machine learning algorithm that groups similar data points by continuously merging the closest clusters step by step. Initially, each data point is treated as an individual cluster, and clusters are merged until all points form a hierarchical tree structure called a dendrogram. Ward linkage method is commonly used to reduce the variance within clusters during merging.

In this project, Agglomerative Hierarchical Clustering was applied on the features TOTAL_SPENDING, FREQUENCY, and Recency to compare its performance with the K-Means clustering algorithm. The dendrogram and clustering results were analyzed to study customer grouping patterns. The model achieved a Silhouette Score of 0.3936, which was lower than the K-Means score of 0.5294, showing that K-Means produced better-defined customer clusters for this dataset.

K-Nearest Neighbors (KNN) Classifier

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and prediction tasks. The algorithm works by finding the k nearest data points to a new input based on distance measures such as Euclidean distance. The new data point is assigned to the class that is most common among its nearest neighbors.

In this project, the KNN classifier was trained using the features AVG_ORDER_VALUE, AVG_PURCHASE_GAP, and Recency to predict customer segments generated by the K-Means clustering model. The value of k was selected as 5, meaning the prediction was based on the five nearest neighboring customers. The model achieved an accuracy of 81.84% on the test dataset and was used to classify new customers into segments such as Casual, VIP, Loyal, and At-Risk customers.

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique used in machine learning and data analysis to reduce the number of features in a dataset while preserving most of the important information and variance present in the data. It converts the original correlated variables into a smaller set of new uncorrelated variables called principal components.

Each principal component is a linear combination of the original features and captures the maximum possible variance in the data. The first principal component captures the highest variance, the second captures the next highest variance while remaining independent of the first, and so on.

The main purpose of PCA is to simplify complex datasets, reduce computational complexity, remove redundancy caused by correlated features, and make data visualization easier.

Working of PCA

1. The data is first standardized so that all features contribute equally.
2. PCA calculates the covariance matrix to measure relationships between features.
3. Eigenvalues and eigenvectors are computed from the covariance matrix.
4. Principal components are selected based on the highest eigenvalues because they capture the maximum variance.
5. The original data is transformed into a lower-dimensional space using these principal components.

Formula for Principal Component:

PC_1=w_1x_1+w_2x_2+\cdots+w_nx_n

Where:
• (PC_1) = first principal component
• (x_1, x_2, ..., x_n) = original features
• (w_1, w_2, ..., w_n) = weights assigned to features

Importance of PCA

• Reduces the number of dimensions in large datasets
• Removes multicollinearity between correlated features
• Improves visualization of high-dimensional data
• Reduces computational cost for machine learning algorithms
• Helps in identifying important patterns and relationships in data
• Makes cluster separation easier to understand visually

In this project, PCA was used to reduce the clustering features into two-dimensional (2D) and three-dimensional (3D) principal components for visualization purposes. The transformed data was plotted using scatter plots to visually analyze the separation between different customer clusters generated by the K-Means algorithm. PCA helped in clearly understanding the distribution, grouping, and separation of customer segments such as Casual, VIP, Loyal, and At-Risk customers.

Elbow Method

The Elbow Method is a technique used to determine the optimal number of clusters (k) in the K-Means clustering algorithm. It helps in selecting the value of k that produces well-formed clusters without unnecessary complexity.

The method works by running the K-Means algorithm multiple times with different values of k and calculating the Within-Cluster Sum of Squares (WCSS) for each value. WCSS measures the total squared distance between data points and their corresponding cluster centroids. Lower WCSS values indicate that the data points are closer to their cluster centers, meaning better clustering.

Formula for WCSS:

WCSS=\sum_{i=1}^{k}\sum_{x\in C_i}(x-\mu_i)^2

Where:
• k = number of clusters
• (C_i) = set of data points in cluster i
• (\mu_i) = centroid of cluster i
• ((x-\mu_i)^2) = squared distance between the data point and centroid

Working of the Elbow Method

1. Select a range of k values, such as 1 to 10.
2. Apply the K-Means algorithm for each value of k.
3. Calculate the WCSS value for every clustering result.
4. Plot the values of k on the x-axis and WCSS on the y-axis.
5. Observe the graph to find the point where the decrease in WCSS starts slowing down significantly. This point forms an “elbow” shape in the graph.
6. The k value at this elbow point is considered the optimal number of clusters because adding more clusters after this point gives only minor improvement.

In this project, the Elbow Method was applied by calculating WCSS values for k ranging from 1 to 10. The resulting graph showed a clear elbow at k=4, indicating that four clusters provided the best balance between clustering performance and model simplicity. Therefore, k=4 was selected as the optimal number of customer clusters for the K-Means model.

Exhaustive Feature Selection

Exhaustive Feature Selection is a process used to identify the best combination of features for building an effective machine learning model. In this method, multiple possible combinations of features are tested systematically, and each combination is evaluated using a performance metric to determine which set of features produces the best results.

In this project, different combinations of customer-related features were generated and tested using the K-Means clustering algorithm. Feature sets containing combinations of 3 to 6 features were evaluated, including attributes such as TOTAL_SPENDING, FREQUENCY, AVG_PURCHASE_GAP, AVG_ORDER_VALUE, PURCHASE_INTENSITY, and Recency. For each feature combination, clustering was performed with k=4, and the Silhouette Score was calculated to measure cluster quality and separation.

After comparing the results of all combinations, the feature set with the highest and most interpretable Silhouette Score was selected for the final clustering model. This process helped in choosing the most meaningful features for accurate customer segmentation and improved the overall clustering performance.

4. Evaluation Metrics Used

Silhouette Score

The Silhouette Score is an evaluation metric used to measure the quality of clustering in machine learning models. It determines how well each data point fits within its assigned cluster compared to other neighbouring clusters. The score helps in analyzing whether the clusters formed are properly separated and compact.

The Silhouette Score ranges from -1 to +1:
• A value close to +1 indicates that the data point is well matched to its own cluster and far from other clusters.
• A value close to 0 indicates overlapping clusters.
• A negative value indicates that the data point may have been assigned to the wrong cluster.

Formula:

s(i)=\frac{b(i)-a(i)}{\max(a(i),b(i))}

Where:
• a(i) = average distance between a data point and all other points within the same cluster (intra-cluster distance)
• b(i) = average distance between the data point and points in the nearest neighbouring cluster

In this project, the Silhouette Score was used to compare the performance of different clustering algorithms and feature combinations. The K-Means clustering model achieved a Silhouette Score of 0.5294, while Agglomerative Hierarchical Clustering achieved 0.3936. Since the K-Means score was higher, it indicated better cluster separation and more effective customer segmentation for the dataset.

Accuracy Score

Accuracy Score is a performance evaluation metric used in classification problems to measure how correctly a machine learning model predicts the output classes. It represents the proportion of correctly classified instances out of the total number of predictions made by the model.

The value of accuracy ranges from 0 to 1, where a value closer to 1 indicates better prediction performance. Accuracy is one of the most commonly used metrics for evaluating classification algorithms.

Formula:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Where:
• Correct Predictions = Number of instances correctly classified by the model
• Total Predictions = Total number of instances tested by the model

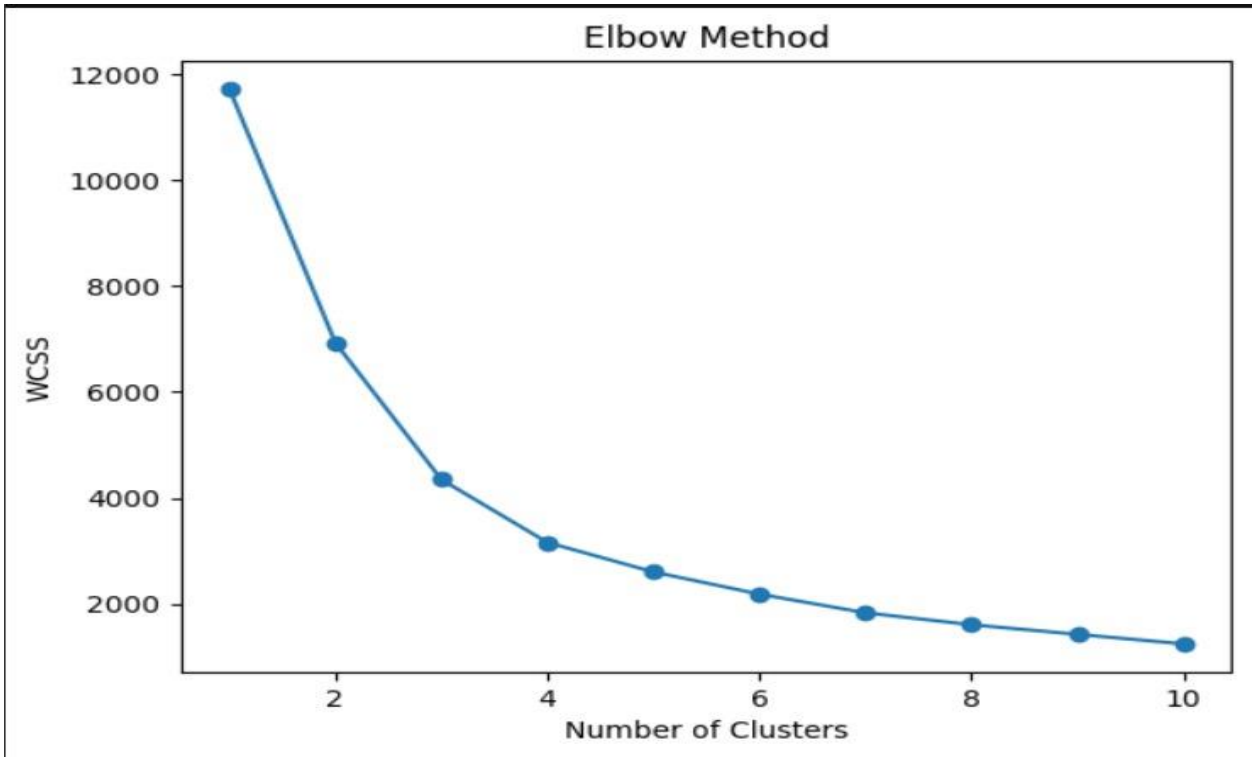
For example, if a model correctly predicts 82 out of 100 test instances, the accuracy would be:

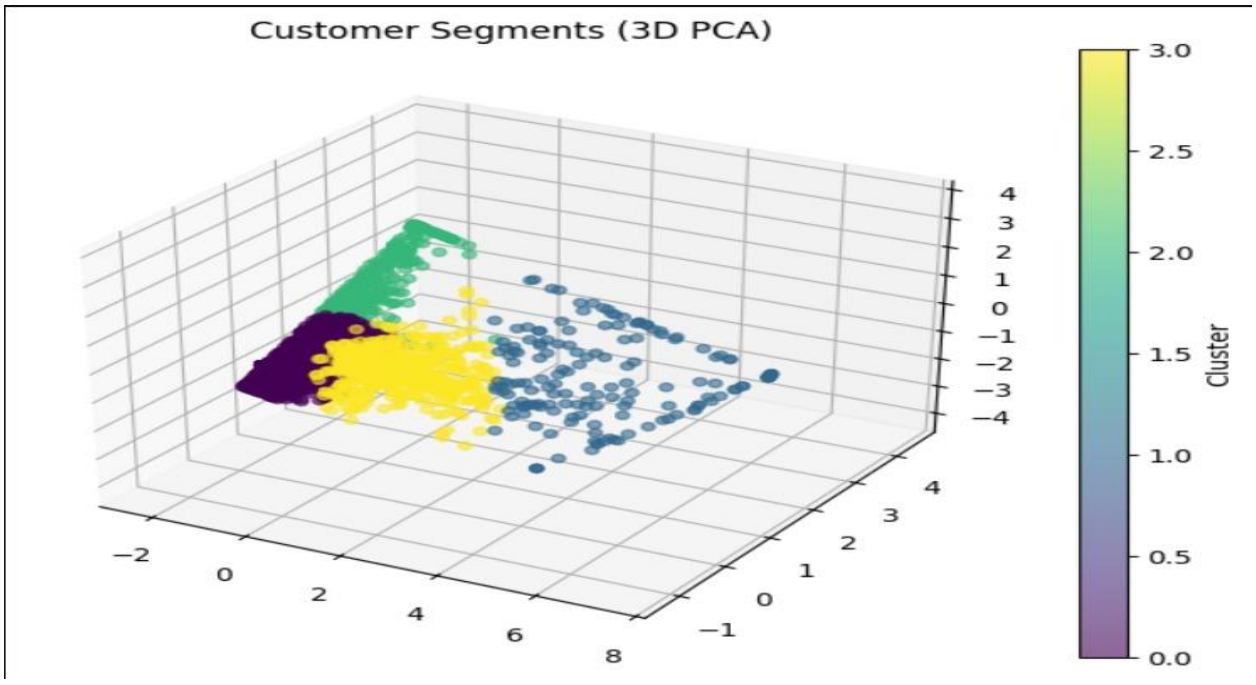
$$\text{Accuracy} = \frac{82}{100} = 0.82 = 82\%$$

In this project, Accuracy Score was used to evaluate the performance of the K-Nearest Neighbors (KNN) classifier for customer segment prediction. The dataset was divided into training and testing sets, and the model predictions on the test data were compared with the actual cluster labels. The KNN classifier achieved an accuracy of 81.84%, indicating that the model correctly predicted customer segments for most of the test instances.

5. Results and Visualizations

Cluster Profiles after K-Means (k=4):





Cluster Analysis and Model Performance

Cluster 0 — Casual Customers

This cluster contains 2,788 customers, representing nearly 71% of the total customer base, making it the largest customer segment. Customers in this group have an average purchase frequency of 2.5 transactions and an average spending of approximately Rs.13,643, with an average purchase gap of 49 days between transactions. These customers shop occasionally and contribute relatively lower revenue compared to other segments. They are generally low-engagement customers who respond better to seasonal offers, discounts, and promotional campaigns. Strategies such as introductory coupons, festival discounts, and personalized advertisements can help increase their purchasing activity.

Cluster 1 — VIP Customers

This segment contains 163 customers, representing about 4% of the total customers, but it is the most valuable and profitable customer group. VIP customers show a very high average purchase frequency of 23.5 transactions and an average spending of approximately Rs.1,73,065. Their average purchase gap is only 16 days, indicating highly frequent engagement with the business. These customers are loyal, premium buyers who contribute a significant portion of the total revenue. Businesses should focus on customer retention strategies for this segment through premium memberships, exclusive offers, early product access, personalized recommendations, and high-value rewards programs.

Cluster 2 — At-Risk Customers

This cluster includes 322 customers, accounting for nearly 8% of the customer base. These customers have an average purchase frequency of only 2.2 transactions and an average spending of around Rs.16,028. Their average purchase gap is approximately 198 days, which indicates long inactivity periods and a high probability of customer churn. Although some customers in this segment previously spent reasonable amounts, their low recent activity suggests declining engagement. Businesses can target this group with win-back campaigns, special discounts, reminder notifications, and personalized retention strategies to encourage re-engagement.

Cluster 3 — Loyal Customers

This segment contains 637 customers, representing around 16% of the customers. Loyal customers have an average purchase frequency of 9.7 transactions and average spending of approximately Rs.66,376, with an average purchase gap of 34 days. These customers regularly interact with the business and generate stable recurring revenue. While they may not spend as aggressively as VIP customers, they form a dependable long-term customer base. Businesses can

strengthen this segment through loyalty programs, reward points, referral benefits, and personalized product recommendations.

Model Performance

The K-Means clustering algorithm achieved a Silhouette Score of 0.5294, indicating good cluster separation and effective customer grouping. In comparison, Agglomerative Hierarchical Clustering achieved a lower Silhouette Score of 0.3936, showing that K-Means produced more compact and well-separated clusters for this dataset.

The K-Nearest Neighbors (KNN) classifier, trained on the generated cluster labels, achieved a test accuracy of 81.84%. This demonstrates that the model can effectively predict customer segments for new customers based on their purchasing behavior and transaction features.

Top Product Categories

Analysis of customer purchasing patterns showed that the most popular product categories among customers were:

- Sports — preferred by 794 customers
- Beauty — preferred by 789 customers
- Gifts — preferred by 743 customers
- Electronics — preferred by 725 customers
- Stationery — preferred by 710 customers

These product categories represent the highest customer engagement and can be prioritized for promotional campaigns, inventory management, and targeted recommendation systems.

6. Conclusion and Future Scope

This project successfully built a complete behavioral segmentation system for 3,910 retail customers. The team engineered ten customer-level behavioral features from raw transaction data, conducted systematic feature selection across 30+ combinations, and compared K-Means against Hierarchical Clustering to select the best-performing algorithm. The primary innovation was training a KNN classifier (81.84% accuracy) on the cluster labels to predict segments for new customers in real time — a capability not present in the base project description. An

interactive interface was built to deliver personalized discount strategies (VIP: 25%, At-Risk: 20%, Loyal: 15%, Casual: 5%) tied to each customer's preferred product category.

In future iterations, the system could be deployed as a REST API using Flask or FastAPI for CRM integration. Real-time POS data ingestion would allow dynamic segment updates. DBSCAN could be explored for anomaly detection. Collaborative filtering could add product-level cross-sell recommendations, and A/B testing could measure the actual revenue uplift from the segmentation strategies.

7. References

1. Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. JMLR 12, 2825–2830 (2011).
2. McKinney, W. Data Structures for Statistical Computing in Python. Proceedings of SciPy 2010.
3. Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow. O'Reilly, 2022.
4. GeeksforGeeks — K-Means Clustering: https://www.geeksforgeeks.org
5. Scikit-learn Documentation — KNeighborsClassifier: https://scikit-learn.org
6. Stack Overflow: https://stackoverflow.com